



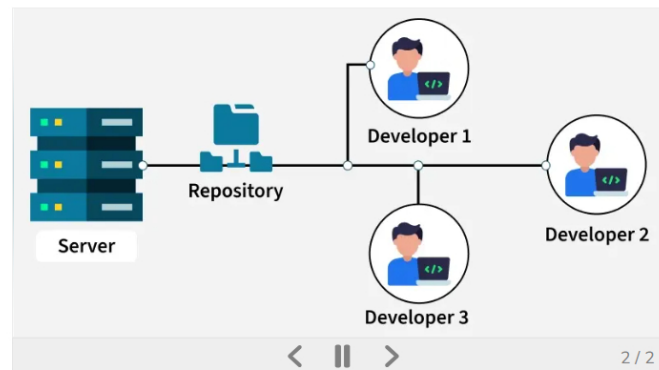
Version Control Systems

Last Updated : 11 Mar, 2026



A Version Control System (VCS) is a tool used in software development and collaborative projects to track and manage changes to source code. It allows developers to:

- Record and track every update to the codebase.
- Collaborate on code without overwriting each other's work.
- Revert to earlier states of the project if needed.
- Maintain a detailed and structured history of the project's evolution.



- The image shows a server connected to multiple developers.
- The repository stores the project files on the server.
- Developers access and update the repository to collaborate on the same project.
- Changes made by one developer can be shared with others through the repository.
- This setup helps maintain a centralized version of the project code.

Components of Version Control Systems

Version Control Systems work using a few core concepts that help teams manage code changes and collaborate efficiently.

- **Repository:** A central location that stores all project files along with their complete change history and metadata like author and commit message.
- **Revision:** A specific saved version of a file or project, identified using a unique ID such as a hash or number.
- **Branch:** A separate copy of the codebase used to develop features or fix bugs without affecting the main code.
- **Merging:** The process of combining changes from one branch into another, which may sometimes require resolving conflicts.
- **Commit:** A snapshot of changes made to the codebase at a specific time, used to track and manage project history.

Types of Version Control Systems

There are three main types of Version Control Systems:

1. Local Version Control Systems (Local VCS)

A Local Version Control System operates entirely on your personal machine without any connection to a remote repository. All changes and version history are stored in a local database on the user's computer.

In this setup, there is typically only a single user, with no collaboration or

In this setup, there is typically only a single user, with no collaboration or sharing of changes. It stores versions in a local repository and does not support teamwork or remote access.

Characteristics:

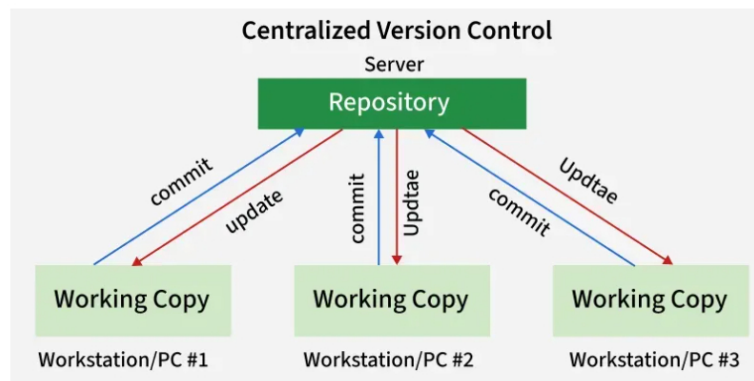
- No internet or server dependency.
- Useful for individual projects.
- Limited to single-user environments.

2. Centralized Version Control Systems

In a Centralized Version Control System, all the files and their version history are stored in a single central server. Developers connect to this server to access or modify files.

The typical workflow is:

1. **Update/Checkout:** A developer retrieves the latest version of the files from the central server.
2. **Make Changes:** The developer works on the files.
3. **Commit:** The developer saves (commits) the changes directly back to the central server, making them immediately available to everyone else.



Pros & Cons:

- The main benefit of Centralized Version Control System (CVCS) is that it makes collaboration among developers while providing visibility into project activities.
- It allows administrators to have fine-grained control over who can do what. It has some downsides as well which led to the development of DVCS.
- The most obvious drawback is the single point of failure represented by the centralized repository. If the repository becomes unavailable, developers cannot collaborate or commit changes.

3. Distributed Version Control Systems

A Distributed Version Control System contains multiple repositories where each user has their own local repository along with a working copy. Simply committing changes does not make them visible to others because a commit only saves changes in the local repository. To share changes with others, they must be pushed to the central repository.

Similarly, to receive changes made by others, you must first pull those changes into your local repository.

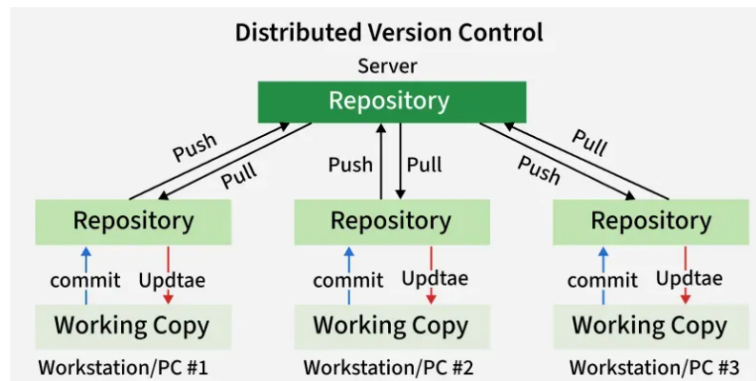
The key difference is the two-step process for sharing changes:

- **Commit:** Saves changes to the local repository (visible only to you).
- **Push:** Uploads committed changes to the central repository so others can access them.

To get changes from others, the process is:

1. **Fetch/Pull:** Downloads the latest changes from the central repository to your local repository.

The most popular distributed version control systems are Git and Mercurial. They help us overcome the problem of single point of failure.



Popular Version Control Systems

Below are five most widely used free VCS tools, perfect for individuals and teams. Each of these version control systems serves different needs, and the choice depends on the project size, team collaboration style and workflow preferences.

Top 5 Free Version Control System



1. Git

Git is the most widely used Distributed Version Control System, developed by Linus Torvalds in 2005 for managing the Linux kernel. It is highly efficient, supports branching and merging, and has a fast, decentralized workflow. Git is the backbone of services like GitHub, GitLab and Bitbucket, making it a popular choice for developers worldwide.

Features of Git:

- Lightweight, fast and efficient.
- Branching and merging are simple and non-destructive.
- Provides powerful commands like git clone, git pull and git push.

2. Subversion (SVN)

Subversion is a popular centralized version control system. While it is not as commonly used in open-source projects today, SVN is still widely used by many organizations and enterprises for its simplicity and centralized structure.

Features of SVN:

- Single central repository.
- Supports branching and tagging but it is less flexible compared to Git.
- Versioning of files and directories.

3. Mercurial

[Mercurial](#) is a distributed version control system similar to Git but with a simpler interface. It is well-suited for both small and large projects and is used in various open-source and enterprise projects.

Features of Mercurial:

- Simple, fast and scalable.
- Supports branching and merging.
- Includes tools for managing project history and changes.

4. CVS (Concurrent Versions System)

CVS is one of the earliest and most influential centralized version control systems. It was widely adopted in the late 1990s and early 2000s and helped shape how modern VCS tools operate. Though largely outdated today, CVS laid the foundation for later tools like Subversion.

Features of CVS:

- Centralized repository architecture.
- Tracks changes to individual files over time.
- Allows multiple developers to work on the same codebase.
- Basic support for branching and tagging, though more limited than modern alternatives.

5. Bazaar

Bazaar is a distributed version control system developed by Canonical, the creators of Ubuntu. Unlike Git or Mercurial, Bazaar supports both centralized and distributed workflows, making it a flexible choice for teams with varied needs. It was widely used in Canonical projects such as Ubuntu and Launchpad.

Features of Bazaar:

- Supports both centralized and distributed version control models.
- Easy to learn and beginner-friendly.
- Human-readable command structure (e.g., `bzr commit`, `bzr push`).
- Cross-platform compatibility (Linux, Windows, macOS).

Version control systems are vital for modern software development, enabling teams or individual developers to track changes, collaborate efficiently, and maintain stable, testable code. Among various types, distributed systems like Git are the most popular due to their flexibility, powerful features, and seamless collaboration.

Suggested Quiz

🔄 6 Questions

In a Local VCS, where are all changes and version history stored?

- ☐ A On a remote server
- ☐ B On GitHub
- ☐ C On the user's personal machine
- ☐ D On a distributed network

[Login to View Explanation](#)

1/6

< Previous **Next >**

Comment

S soumy...

51



Article Tags:

Git

GitHub

Git-Questions

Explore

Git Introduction



Git Installation and Setup



All Git Commands



Most Used Git Commands



Git Branch



Git Merge



Git Tools and Integration



Git Remote Repositories



Collaborating with Git



Advanced Git Commands



Git Workflows



Git Hooks



Managing Large Git Repositories



Troubleshooting Git



How to Generate Public SSH or GPG Keys Using Git Commands?

2 min read

Git Interview Preparation



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us

Legal

Privacy Policy

Contact Us

Advertise with us

GFG Corporate

Solution

Campus Training

Program

Explore

POTD

Job-A-Thon

Blogs

Nation Skill Up

Tutorials

Programming

Languages

DSA

Web Technology

AI, ML & Data

Science

DevOps

CS Core Subjects

Interview

Preparation

Software and Tools

Courses

ML and Data

Science

DSA and

Placements

Web Development

Programming

Languages

DevOps & Cloud

GATE

Trending

Technologies

Videos

DSA

Python

Java

C++

Web Development

Data Science

CS Subjects

Preparation Corner

Interview Corner

Aptitude

Puzzles

GfG 160

System Design